Custom Gameboy SoC V1

V1 Goal

Implement a simple black-and-white, no-audio, Game Boy-inspired SoC on the Basys 3 FPGA board. The system will use a Cal Poly OTTER 32-bit RISC-V CPU to execute bare-metal Snake from a ROM initialization file, with memory-mapped graphics and controller input.

Overall Project Goal

Develop a modular RISC-V-based SoC platform for exploring computer architecture and hardware/software co-design. Progressively introduce custom peripherals and hardware accelerators to solve practical computing problems and study the tradeoffs between software and dedicated hardware implementations.

Basys 3 baord:

- 33,280 logic cells in 5200 slices (each slice contains four 6-input LUTs and 8 flip-flops);
- 1,800 Kbits of fast block RAM;
- Five clock management tiles, each with a phase-locked loop (PLL);
- 90 DSP slices;
- Internal clock speeds exceeding 450 MHz;
- On-chip analog-to-digital converter (XADC).

MMIO:

Address(HEX)	Descritpion	Total Spaces

V1 Modules

Top Level

Inputs:

- Basys Board Clock(Should be 100Mhz)
- Basys Board Middle Button
- Basys Board Buttons
- Basys Board switches

Outputs:

- VGA Stuff

Holds all of the interconnects between modules

CPU

Inputs:

- Clock(Should be 50Mhz)
- reset
- memory read data

Outputs:

- memory address
- memory write data
- memory write signal
- memory read signal
- mem byte bits(to read/write one-4 bytes)

InterConnect

“Made by Gemenai”

We could probably just mux or direct wirte the data to each module and only use a mux to deterimine which is valid, however for now we will use this template

module system_interconnect (
// --- CPU Master Side (Inputs from CPU, Outputs back to CPU) ---
input logic [31:0] cpu_addr,
input logic [31:0] cpu_wdata,
input logic   cpu_read,
input logic   cpu_write,
input logic [3:0] cpu_byte_en,
output logic [31:0] cpu_rdata,   // Muxed data coming back to the CPU

```
// --- Slave 1: IMEM / ROM ---
output logic [31:0] imem_addr,
output logic        imem_read,          // Usually just tied high or driven by cpu_read f
or ROM
input  logic [31:0] imem_rdata,

// --- Slave 2: DMEM / RAM ---
output logic [31:0] dmem_addr,
output logic [31:0] dmem_wdata,
output logic        dmem_read,
output logic        dmem_write,
output logic [3:0]  dmem_byte_en,
input  logic [31:0] dmem_rdata,

// --- Slave 3: PPU (VRAM / Registers) ---
output logic [11:0] ppu_addr,           // Might be a smaller address window
output logic [31:0] ppu_wdata,
output logic        ppu_read,
output logic        ppu_write,
output logic [3:0]  ppu_byte_en,
input  logic [31:0] ppu_rdata,

// --- Slave 4: Joypad / Peripherals (Future Expansion) ---
output logic        joypad_read,
input  logic [31:0] joypad_rdata
```

);

Graphics

The graphics are primarily created by the ROM, CPU, PPU, and VGA controller. The ROM holds information from the 3 layers(Background, Window, Sprites), and the CPU determins what goes where. Once known, the VRAM(usually in PPU) will hold the information of each pixel/tiles, which was written from the CPU from the ROM. The PPU and VGA work together to place each pixel from top-bottom, left-right, on the monitor.

- Assuming 640×480 pixels at 60 Hz

PPU

Inputs:

- Clock for VRAM(Proba bly 50 MHz)
- Clock for PPU(GOtta do calculation)
- Reset
- VRAM Address
- VRAM Write Data
- VRAM byte SEL
- VRAM write enable
- Pixel X from VGA Cont
- Pixel Y from VGA Cont
- Video bit(On/Off)

Outputs:

- VRAM Read Data
- VGA Pixel Data

VGA Controller

Inputs:

- Clock(do the calc, same as PPU)
- reset

Outputs:

- VGA vertical Sync Pulse
- VGA Horizontal Sync Pulse(IDK what both these are lol)
- Pixel X location
- Pixel Y location
- On/Off boolean

Design Decisions

Topic	Decision	Date
DMEM and IMEM Harvard Arch	Unlike the original OTTER memory organization, IMEM and DMEM will be implemented as separate memories. Instruction fetches access IMEM, while CPU loads and stores access DMEM. This simplifies the memory interfaces and allows the two memories to be implemented independently using FPGA BRAM. This is with many assumptions such as the divided clock which allows us read and write without a cycle of latency, however we may find problems in the future and can use the dual port again if needed.	8/21/26

Dual Port VRAM/PPU	The CPU and PPU may need to access VRAM concurrently. VRAM will therefore use dual-port BRAM, allowing the CPU to read/write through one port while the PPU independently reads framebuffer data through the other port. CPU PORT NEEDS R/W. PPU only R.	8/21/26